

Introduction to Python

What is Python?

Python is a powerful and widely-used programming language that debuted in 1991. While Python 3 is the current standard, you might encounter Python 2 code in older resources. The language offers flexibility in how you interact with it - you can choose from various interfaces, and run multiple independent instances (called **kernels**) simultaneously on your computer.

One important detail to remember: Python is case **sensitive**. For example, the built-in function `type` (lowercase) tells you what kind of object you're working with, but writing it as `Type` will trigger an error because Python treats uppercase and lowercase letters as distinct characters.

Setting up Python

To write and run Python code, you need two things: a Python installation and an interface to work in. You have several options, from fully local setups to cloud-based environments:

Option 1: Anaconda (easiest local setup). Download and install from anaconda.com. Anaconda comes pre-packaged with Python and most data science libraries used in this course. After installing, open **Anaconda Navigator** and start **Jupyter Notebook** from there. You're ready to go. For those who prefer the command line, you can bypass the Navigator and launch tools directly from the Terminal (Mac) or Anaconda Prompt (Windows).

Option 2: uv (lightweight, modern). If you're comfortable with the command line, **uv** (`docs.astral.sh/uv`) is a fast, modern tool for managing Python projects. To get started:

1. Install uv (once): `curl -LsSf https://astral.sh/uv/install.sh | sh` (Mac/Linux) or `powershell - ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"` (Windows)
2. Create a project environment: `uv init myproject && cd myproject`
3. Add packages: `uv add numpy pandas matplotlib scikit-learn jupyter`
4. Launch Jupyter: `uv run jupyter notebook`

Option 3: Google Colab (no installation). Go to colab.google with a Google account. Colab is a cloud-hosted notebook environment — it runs on Google's servers, so you don't need to install anything. Most packages are pre-installed. However, Colab has limitations (e.g. file persistence, custom package versions, some libraries won't work) so it won't be suitable for everything in this course.

Jupyter notebooks

Whichever option you choose, you will be working with **Jupyter notebooks**. The Jupyter ecosystem introduces the **notebook** paradigm, allowing seamless integration of code, output, and documentation in a single document. The page that you are reading right now is a rendered notebook.

Historically, developers used **IPython** (Interactive Python), which enhanced the basic Python shell with extra features. IPython introduced **magic commands**, denoted by `%cmd` (like `%cd` for changing directories). While these commands aren't part of Python itself, they're still referenced in many tutorials. For a complete list, use `%quickref`.

IDEs

Beyond notebooks, you may encounter full **Integrated Development Environments (IDEs)** like **Visual Studio Code** or **Cursor**. These provide comprehensive development tools like consoles, text editors, debuggers, and variable inspectors. Feel free to use any tool you're comfortable with, though our instructions will assume you're using Jupyter Notebook or Jupyter Lab. The concepts transfer to other interfaces with minimal adaptation.

Typing Python

Code is typed into code cells like the one below, when you enter that cell and press Shift+Return the code will run and you will get the output underneath. Python returns either an output (preceded by [1]:), a (typically long and difficult) error message, or no answer at all. Here is a super simple example

```
In [42]: l = ["Prachi", "Henar", "Dhruv", "Enric"]
```

```
l[-2:][::-1]
```

```
Out[42]: ['Enric', 'Dhruv']
```

```
In [44]: d = {
    "Prachi" : "+34 123456789",
    "Henar" : "+34 987654321",
    "Dhruv" : "+34 555555555",
    "Enric" : "+34 111111111"
}

d["Enric"]
```

```
Out[44]: '+34 111111111'
```

```
In [12]: type(a)
```

```
Out[12]: str
```

So, if you enter $2 + 2$, the output will be the result of this calculation. But, when you want to store this result for later use (in the same session), you enter it with a name, as follows:

```
In [44]: a = 2 + 2
```

This creates the **variable** `a`. Note that the value of $2 + 2$ is not outputted now. But you can call it:

```
In [45]: a
```

```
Out[45]: 4
```

In Python, when you assign a value to a variable that has already been created, the previous assignment is forgotten. So:

```
In [46]: a = 7 - 2
a
```

```
Out[46]: 5
```

If you copypaste in the console code chunks that you edit in a text editor, you can input several code lines at once. In that case, you will only get the output for the last line. If the cursor is not at the end of the last line, you have to press Shift+Return to get the output. Here is a simple example:

```
In [47]: b = 2 * 3
b - 1
b**2
```

```
Out[47]: 36
```

Note. You would probably have written b^2 for the square of 2, but the caret symbol (^) plays a different role in Python.

Python Packages

When you start a new kernel, only a core set of Python resources is initially available. You'll need to import additional packages for most data science tasks, as these core resources are intentionally kept minimal. These extra resources come in **packages**, which themselves are split into different **modules**. Python can be extended by more than 200,000 packages. Some large packages, like scikit-learn, are collections of modules called **subpackages**.

For example, if you need to calculate the square root of 2, you would import the `math` package, which includes various mathematical functions. Once imported, all its functions become available. The notation `math.sqrt` indicates that `sqrt` is a function from the `math` package.

```
In [48]: import math
math.sqrt(2)
```

```
Out[48]: 1.4142135623730951
```

Alternatively, you can import only the functions that you plan to use:

```
In [49]: from math import sqrt
sqrt(2)
```

```
Out[49]: 1.4142135623730951
```

The following example generates a random number according to a normal distribution by calling a function in the `random` package of `numpy` module.

```
In [50]: import numpy.random as rnd
         rnd.randn()
```

```
Out[50]: 0.8767069099992678
```

Installing packages

Packages must be installed on your computer before they can be imported, and these imports only last for the current kernel session. The `math` package, being part of the **Python Standard Library**, is always available by default.

If you use Anaconda, most packages used in this course are already available and can be directly imported. If you need an extra package, open a terminal and run `pip install packagename`. If you use uv, run `uv add packagename` instead. On Google Colab, most common packages are pre-installed; for others you can run `!pip install packagename` directly in a notebook cell. You may also encounter references to **conda** in tutorials and documentation, it works very similar to pip: `!conda install packagename`.

This course approaches Python from a data science perspective, we therefore focus on its four core packages:

- **NumPy** (1995): Provides support for large vectors and matrices (called **arrays**) - the foundation for scientific computing in Python
- **Matplotlib**: Built on NumPy, this is Python's primary visualization library
- **Pandas**: A powerful data management library inspired by R, built on top of NumPy and Matplotlib. Its introduction has significantly increased Python's popularity in data science, making Python/Pandas a common alternative to R
- **scikit-learn** (2008): A comprehensive machine learning library that works with both NumPy arrays and Pandas objects (though it always returns NumPy arrays)

Data Types

The **data types** in Python are similar to those of other languages. The type can be learned with the function `type`. The main data types are:

- First, we have **integer numbers** (type `int`). There are subdivisions of this basic type, such as `int64`, but you don't need to know about that to start your Python trip.
- We can also have **floating-point** numbers (type `float`), that is, numbers with decimals. We also have subdivisions here, such as `float64`.
- Under type `bool`, Python admits **Boolean** values, which are either `True` or `False`. In Python, Boolean variables are converted to type `int` or `float` by applying a mathematical operator.
- Besides numbers, we can also manage **strings**, with type `str`. Strings come in Python with many methods attached. They will be discussed, in the Pandas context, in a specific chapter.
- Python also has type `datetime` for dealing with **dates and times**. This will also be discussed later in a specific chapter.

Numeric types

As in other languages, data can have different **data types** in Python. The data type can be learned with the function `type`. Let me start with the numeric types. For the variable a defined above:

```
In [51]: type(a)
```

```
Out[51]: int
```

So, a has type int (meaning integer). Another numeric type is that of **floating-point** numbers (`float`), which have decimals:

```
In [52]: b = math.sqrt(2)
         type(b)
```

```
Out[52]: float
```

There are subdivisions of these two basic types (such as `int64`), but I skip them in this brief tutorial. Note that, in Python, integers are not, as in the mathematics textbook, a subset of the real numbers, but a different type:

```
In [53]: type(2)
```

```
Out[53]: int
```

```
In [54]: type(2.0)
```

```
Out[54]: float
```

In the above square root calculation, `b` got type `float` because this is what the `math` function `sqrt` returns. The functions `int` and `float` can be used to convert numbers from one type to another type (sometimes at a loss):

```
In [55]: float(2)
```

```
Out[55]: 2.0
```

```
In [56]: int(2.3)
```

```
Out[56]: 2
```

Boolean Data

We also have **Boolean** (`bool`) variables, whose value is either `True` or `False`:

```
In [57]: d = 5 < a
d
```

```
Out[57]: False
```

```
In [58]: type(d)
```

```
Out[58]: bool
```

Even if they don't appear explicitly, Booleans are always under the hood. When you enter an expression involving a comparison such as `5 < a`, the Python interpreter evaluates it, returning either `True` or `False`.

Here, I have defined a variable by means of such an expression, so I got a Boolean variable. Warning: as a comparison operator, equality is denoted by two equal signs. This may surprise you.

```
In [59]: a == 4
```

```
Out[59]: False
```

Boolean variables can be converted to `int` and `float` type with the functions mentioned above, but also by applying a mathematical operator:

```
In [60]: math.sqrt(d)
```

```
Out[60]: 0.0
```

```
In [61]: 1 - d
```

```
Out[61]: 1
```

Strings

Besides numbers, we can also manage **strings** with type `str`:

```
In [62]: c = 'Lovelace'
type(c)
```

```
Out[62]: str
```

The quote marks indicate type `str`. You can use single or double quotes, but take care of using the same on both sides of the string. Strings come in Python with many methods attached. These methods will be discussed later in this course

Data Containers

Lists

Python has several types for objects that work as **data containers**. The most versatile is the **list**, which is represented as a sequence of comma-separated values inside square brackets.

Lists can contain items of different type. A simple example of a list, of length 4, follows.

```
In [63]: mylist = ['Lovelace', 'Babbage', 'Shannon', 'Turing']
```

```
In [64]: len(mylist)
```

```
Out[64]: 4
```

Lists can be concatenated in a very simple way in Python:

```
In [65]: newlist = mylist + [2, 3]
newlist
```

```
Out[65]: ['Lovelace', 'Babbage', 'Shannon', 'Turing', 2, 3]
```

Now, the length of `newlist` is 6:

```
In [66]: len(newlist)
```

```
Out[66]: 6
```

The first item of `mylist` can be extracted as `mylist[0]`, the second item as `mylist[1]`, etc. The last item can be extracted either as `mylist[3]` or as `mylist[-1]`. Sublists can be extracted by using a colon inside the brackets, as in:

```
In [67]: mylist[0:2]
```

```
Out[67]: ['Lovelace', 'Babbage']
```

Note that `0:2` includes `0` but not `2`. This is a general rule for indexing in Python. Other examples:

```
In [68]: mylist[2:]
```

```
Out[68]: ['Shannon', 'Turing']
```

```
In [69]: mylist[:3]
```

```
Out[69]: ['Lovelace', 'Babbage', 'Shannon']
```

```
In [70]: mylist[:-1]
```

```
Out[70]: ['Lovelace', 'Babbage', 'Shannon']
```

The items of a list are ordered, and can be repeated. This is not so in other data containers.

Set

A **set** is represented in the same way as a list, but with curly braces replacing the square brackets:

```
In [71]: myset = {'Messi', 'Cristiano', 'Neymar', 'Coutinho'}
```

A difference between the list and the set is that the elements of a set are not ordered, and repetition is ignored.

Tuple

A **tuple** is like a list, represented with parentheses instead of square brackets. Unlike lists, tuples are **immutable** - once created, their elements cannot be changed. You'll encounter tuples frequently as return values from functions (for example, NumPy's `.shape` attribute returns a tuple):

```
In [72]: mytuple = ('Messi', 'Cristiano', 'Neymar', 'Coutinho')
```

Ranges

A **range** is a sequence of integers which in many aspects works as a list, but the terms of the sequence are not saved as in a list. Instead, only the procedure to create the sequence is saved. The syntax is `range(start, end, step)`. Example:

```
In [73]: myrange = range(0, 10, 2)
list(myrange)
```

```
Out[73]: [0, 2, 4, 6, 8]
```

Note that the items from a range cannot be printed directly. So, I have converted the range to a list here with the function `list`. If the step is omitted, it is assumed to be 1:

```
In [74]: list(range(5, 12))
```

```
Out[74]: [5, 6, 7, 8, 9, 10, 11]
```

If the start is also omitted, it is assumed to be 0:

```
In [75]: list(range(10))
```

```
Out[75]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Dictionaries

A dictionary is a set of **pairs key/value**. For instance, the following dictionary contains three features of an individual:

```
In [76]: my_dict = {'name': 'Joan', 'gender': 'F', 'age': 32}
```

The keys can be listed:

```
In [77]: my_dict.keys()
```

```
Out[77]: dict_keys(['name', 'gender', 'age'])
```

In the dictionary, a value is not extracted using an index which indicates its order in a sequence, as in the list, but using the corresponding key:

```
In [78]: my_dict['name']
```

```
Out[78]: 'Joan'
```

Example

Dictionaries are important because a lot of real life data comes in the form of dictionaries. Let's say you want to get the latests information on bitcoin. You may want to call the [CoinGecko](https://api.coingecko.com/) API (Application Programming Interface) to obtain that data.

Typically, these API's are essentially just websites that you visit. In this case:

[https://api.coingecko.com/api/v3/coins/markets?](https://api.coingecko.com/api/v3/coins/markets?vs_currency=usd&ids=bitcoin&order=market_cap_desc&per_page=100&page=1&sparkline=false&locale=en)
[vs_currency=usd&ids=bitcoin&order=market_cap_desc&per_page=100&page=1&sparkline=false&locale=en](https://api.coingecko.com/api/v3/coins/markets?vs_currency=usd&ids=bitcoin&order=market_cap_desc&per_page=100&page=1&sparkline=false&locale=en)

```
In [48]: import requests
```

```
api_url = 'https://api.coingecko.com/api/v3/coins/markets?vs_currency=usd&ids=bitcoin&order=market_cap_desc&per_page=100&page=1&sparkline=false&locale=en'
data = requests.get(api_url).json()[0]
```

```
data
```

```
Out[48]: {'id': 'bitcoin',
'symbol': 'btc',
'name': 'Bitcoin',
'image': 'https://coin-images.coingecko.com/coins/images/1/large/bitcoin.png?1696501400',
'current_price': 72141,
'market_cap': 1443914041810,
'market_cap_rank': 1,
'fully_diluted_valuation': 1443914041810,
'total_volume': 39127135466,
'high_24h': 72888,
'low_24h': 70573,
'price_change_24h': 944.05,
'price_change_percentage_24h': 1.32597,
'market_cap_change_24h': 19915665097,
'market_cap_change_percentage_24h': 1.39857,
'circulating_supply': 20013912.0,
'total_supply': 20013912.0,
'max_supply': 21000000.0,
'ath': 126080,
'ath_change_percentage': -42.78139,
'ath_date': '2025-10-06T18:57:42.558Z',
'atl': 67.81,
'atl_change_percentage': 106288.77056,
'atl_date': '2013-07-06T00:00:00.000Z',
'roi': None,
'last_updated': '2026-04-10T12:10:28.145Z'}
```

```
In [47]: data['current_price']
```

```
Out[47]: 72166
```

```
In [49]: data['current_price']
```

```
Out[49]: 72141
```

Functions

A **function** takes a collection of **arguments** and performs an action. Let me present a couple of examples of value-returning functions. They are easily distinguished from other functions, because the definition's last line is a `return` clause.

A first example follows. Note the **indentation** after the colon.

```
In [54]: def f_mathy_stuff(x):
        y = 1/(1 - x**2)
        return y

f_mathy_stuff(0.5)
```

```
Out[54]: 1.3333333333333333
```

```
In [58]: # A = P (1+r/n)**(n*t)

def compound_interest(P, r, t):
    A = P * (1+r)**(t)
    return A

compound_interest(100000, 0.072, 10)
```

```
Out[58]: 200423.13616543383
```

When we define a function, Python just takes note of the definition, accepting it when it is syntactically correct (parentheses, commas, etc). The function can be applied later to different arguments.

```
In [81]: f(2)
```

```
Out[81]: -0.3333333333333333
```

If we apply the function to an argument for which it does not make sense, Python will return an error message which depends on the values supplied for the argument.

```
In [82]: f(1)
```

```

-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[82], line 1
----> 1 f(1)

Cell In[80], line 2, in f(x)
      1 def f(x):
----> 2     y = 1/(1 - x**2)
      3     return y

ZeroDivisionError: division by zero

```

```
In [ ]: f('Mary')
```

Short functions can be defined on a single line:

```
In [ ]: def g(x): return x**2
g(3)
```

Sometimes, you will see them written in an even shorter way called **lambda functions**:

```
In [ ]: g = lambda x : x**2
g(3)
```

Functions can have more than one argument, as in:

```
In [ ]: def g(x, y): return x*y/(x**2 + y**2)
g(1, 1)
```

Loops and conditional logic

Loops are used in practically all programming languages, so, if you have seen them if you have any kind of experience in programming. In particular, **for** loops are used to avoid repetition. Suppose that you wish to extract the first letter of the names of the list `mylist`, storing them in a new list. You can use the **for** loop to iterate the extraction:

```
In [ ]: inilist = [name[0] for name in mylist]
inilist
```

Loops are much less frequent in the data science practice, because NumPy and Pandas provide **vectorized functions**, that, when applied to a data container such as a Pandas series, return a data container with same shape, whose terms are the values of the function on the corresponding terms of the original data container. Nevertheless, we may use occasionally a **for** loop in this course.

Also ubiquitous in programming is **conditional logic**, operationalized through **if-then-else** commands. You also have this in Python. For instance, if you wish to create a dummy flag for names with more than 5 letters in the list `mylist`, you can do it with:

```
In [ ]: flaglist = [1 if len(name) > 5 else 0 for name in mylist]
flaglist
```

It is also rare to find explicit if-the-else arguments in data science, since "vectorial" syntax is preferred (and typically leads to a faster execution).

You try it

a) Create a phone book using a `dict` where you have 5 names of your friends as a key and their phone number as the values.

b) Then, also create a reverse mapping so that you can find out who's calling you based on the number.

c) Define a function called `find_caller_id` that returns the name of the person who's calling you based on the number, use the reverse mapping.

```
In [ ]: phonebook = ...
```

Asking ChatGPT for help

You are allowed to use ChatGPT or other LLMs in this course. However, I recommend using them as a complement to your own understanding, not as a substitute. When exploring a feature of Python that you are not familiar with, make sure to look up the relevant libraries to verify that you're using them correctly. Here the advice will be general to ensure it works with every LLM.

Prompt Suggestions

Don't ask for general prompts, instead ask specific questions and show ChatGPT some data so it knows what you're dealing with. These examples contain concepts and libraries that we didn't cover yet at this point but the general style of asking tends to work for problems encountered in this course.

Example 1:

Prompt:

I want you to act as a pro coder. Please write me a regex in Python that extracts the email from the following pandas Series:

```
s = pd.Series('Enric enric@iese.edu', 'Franz franz@iese.edu').
```

Response:

Here's a Python code snippet using pandas and regular expression to extract emails:

```
In [ ]: import pandas as pd
import re

s = pd.Series(['Enric <enric@iese.edu>', 'Franz <franz@iese.edu>'])
email_pattern = r'<(.*?)>'
s_extracted = s.str.extract(email_pattern)

print(s_extracted)
```

This will output a new Series with just the email addresses.

Note how it (a) corrected a bug in my original code, but (b) introduced a redundancy as it's importing a package (`re`) that it's not using ...

Example 2

Prompt:

I have the following dataframe called `df_aapl`, plot the rolling average of the Open price over time using just pandas, no matplotlib or numpy. Make it look professional and add a title:

	Open	High	Low	Close	Adj Close	Volume	
2023-08-01			196.240005	196.729996	195.279999	195.610001	
	195.346207		35175100				
2023-08-02			195.039993	195.179993	191.850006	192.580002	
	192.320297		50389300				
2023-08-03			191.570007	192.369995	190.690002	191.169998	
	190.912201		61235200				

Response:

You can plot the rolling average of the "Open" price using Pandas directly like this:

```
In [ ]: import pandas as pd

data = {
    'Open': [196.24, 195.039993, 191.570007],
    'High': [196.729996, 195.179993, 192.369995],
    'Low': [195.279999, 191.850006, 190.690002],
    'Close': [195.610001, 192.580002, 191.169998],
    'Adj Close': [195.346207, 192.320297, 190.912201],
    'Volume': [35175100, 50389300, 61235200]
}
index = pd.to_datetime(['2023-08-01', '2023-08-02', '2023-08-03'])
df_aapl = pd.DataFrame(data, index=index)

df_aapl['Open'].rolling(window=2).mean().plot(title='Rolling Average of Open Price', grid=True);
```

The code works but as it didn't know your data it made up its own, simply replace data with your own variable name and you should be good to go.

Additional Resources

There are many books for learning about Python, but some of them would not be appropriate for learning how to work with data in Python. It can even happen that you do not find anything about data in many of them. Mind that Python has so many applications that the intersection of the know-how of all Python users is relatively narrow. For an introduction to Python as a programming language, in a computer science context, I would recommend Zelle (2010). For the self-learning data scientist, McKinney (2017) and VanderPlas (2017) are both worth their price.

There is also plenty of learning materials in Internet, including MOOC's. For instance, **Coursera** has a pack of courses on Python (see coursera.org/courses?query=python). But, probably, the most attractive marketplace for data science courses is **DataCamp**. They offer, under subscription or academic license, an impressive collection of courses, most of them focused on either R or Python. In addition to follow DataCamp courses, you can also benefit from the **DataCamp Community Tutorials**, which are free and cover a wide range of topics. Finally, a good place to start is [CS dojo](#).

References

This document is based on an original version by Miguel Ángel Canela, 2022.